



UNIVERSIDAD DE CARABOBO
FACULTAD DE INGENIERIA
MAESTRIA EN MATEMÁTICAS Y COMPUTACION

ASIGNACION TRES

Compendio sobre Teoría de Grafos y Algoritmos de Enrutamiento: Desde los Fundamentos Clásicos hasta el Estado del Arte Topológico.

Autor: Ing. Melvin E. Padilla
Cátedra: Diseño de Algoritmos
Profesor: Ing. Alejandro Bolívar

Valencia, Junio de 2026

Resumen

Este reporte presenta una revisión exhaustiva y rigurosa de la teoría de grafos y los algoritmos de enrutamiento optimizado. El análisis abarca desde las definiciones topológicas elementales y las representaciones matriciales computacionales hasta las metodologías contemporáneas de aprendizaje profundo en grafos (Graph Neural Networks). Se exploran en profundidad los algoritmos clásicos de caminos mínimos, flujos en redes, análisis de centralidad y optimización combinatoria, proporcionando formulaciones matemáticas explícitas, análisis de complejidad algorítmica y comparaciones computacionales. El objetivo fundamental es consolidar un marco de referencia analítico para el diseño, simulación y optimización de redes complejas e infraestructuras industriales.

Índice

1. Introducción y Fundamentos Topológicos	4
1.1. Definiciones Formales	4
1.2. Clasificación de Estructuras	4
1.3. Representaciones Computacionales	4
1.3.1. Matriz de Adyacencia	5
1.3.2. Lista de Adyacencia	5
1.3.3. Matrices Dispersas (Sparse Matrices)	5
2. Algoritmos Clásicos de Caminos Mínimos y Exploración	5
2.1. Algoritmo de Dijkstra	5
2.1.1. Inicialización	6
2.1.2. Extracción del Mínimo	6
2.1.3. Fase de Relajación	6
2.1.4. Implementación en Python (Estructura de Datos Pura)	6
2.1.5. Análisis del Caso Práctico	8
2.2. Algoritmo A* (A-Star)	8
2.2.1. Ampliación Teórica: Anatomía de la Ecuación $f(n) = g(n) + h(n)$	8
2.2.2. Heurísticas más comunes en ingeniería	9
2.2.3. Implementación en Python (Ruteo Espacial en Cuadrícula)	9
2.2.4. La Diferencia de Rendimiento	11
2.3. Algoritmo de Bellman-Ford	11
2.3.1. Ampliación Teórica: ¿Por qué falla Dijkstra y cómo lo soluciona Bellman-Ford?	12
2.3.2. Implementación en Python	12
2.3.3. Análisis del Resultado	14
2.3.4. Implementación en Python con NetworkX	15
2.3.5. ¿Por qué hacerlo así?Eficiencia y Mantenibilidad:	16
3. Árboles de Expansión Mínima y Flujo en Redes	16
3.1. Árbol de Expansión Mínima (MST)	16
3.1.1. Ampliación Teórica: Prim vs. Kruskal	17
3.1.2. Algoritmo de Prim (Expansión Radial)	17
3.1.3. Algoritmo de Kruskal (Construcción Global)	17
3.1.4. Implementación en Python (Algoritmo de Kruskal)	17
3.1.5. Análisis del Resultado Algorítmico	19
3.2. Algoritmos de Flujo Máximo	20
3.2.1. Ampliación Teórica: El Paradigma de la Capacidad	20
3.2.2. El Teorema del Flujo Máximo - Corte Mínimo (El Cuello de Botella)	20
3.2.3. Implementación en Python (NetworkX)	21

3.2.4. Análisis Estructural del Sistema	22
4. Análisis de Centralidad y Optimización Combinatoria	23
4.1. Métricas de Centralidad	23
4.1.1. Ampliación Teórica: Desglosando las Métricas	23
4.1.2. Centralidad de Intermediación (Betweenness Centrality)	23
4.1.3. PageRank (Dominancia por Influencia)	24
4.1.4. Implementación en Python (NetworkX)	24
4.1.5. Análisis del Comportamiento	25
5. El Estado del Arte: Aprendizaje Profundo en Grafos (GNN)	26
5.0.1. Desglosando la Ecuación Matemática (Paso de Mensajes)	26
5.0.2. Implementación en Python: Construyendo la Ecuación 7 desde Cero	27
5.0.3. ¿Qué está sucediendo aquí físicamente?	28

1. Introducción y Fundamentos Topológicos

La teoría de grafos constituye el pilar formal para el modelado, análisis y optimización de sistemas interconectados. Estructuras como redes de distribución eléctrica, sistemas de automatización industrial, redes logísticas y sistemas de transporte pueden ser completamente caracterizadas mediante estas herramientas matemáticas discretas.

1.1. Definiciones Formales

Formalmente, un grafo se define como un par ordenado $G = (V, E)$, donde:

- V es un conjunto no vacío de **vértices** o **nodos**, que representan las entidades del sistema (e.g., subestaciones, motores, servidores, nodos de control).
- E es un conjunto de **aristas**, **arcos** o **enlaces**, que representan las relaciones o conexiones físicas/lógicas entre dichas entidades, de tal manera que $E \subseteq \{\{u, v\} \mid u, v \in V\}$.

1.2. Clasificación de Estructuras

Dependiendo de la naturaleza de las interconexiones, los grafos se clasifican estructuralmente en:

1. **Grafos No Dirigidos:** Las aristas no poseen una orientación intrínseca; la relación entre u y v es simétrica ($(u, v) \equiv (v, u)$).
2. **Grafos Dirigidos (Digrafos):** Las aristas poseen una dirección definida, representadas como pares ordenados (u, v) donde el flujo u operación se origina en u (nodo cola) y termina en v (nodo cabeza).
3. **Grafos Ponderados:** Cada arista $e \in E$ tiene asignada una función de costo o peso $w(e) \in \mathbb{R}$. En entornos de ingeniería, este peso representa magnitudes físicas o financieras medibles, tales como distancia en metros, caída de tensión, resistencia óhmica, latencia de red o costos de construcción.

1.3. Representaciones Computacionales

La eficiencia de los algoritmos de enrutamiento depende críticamente de la estructura de datos subyacente utilizada para almacenar el grafo en memoria:

1.3.1. Matriz de Adyacencia

Para un grafo con $|V| = n$, la matriz de adyacencia A es una matriz de dimensiones $n \times n$ donde el elemento A_{ij} se define como:

$$A_{ij} = \begin{cases} w(v_i, v_j) & \text{si } (v_i, v_j) \in E \\ 0 \text{ o } \infty & \text{si } (v_i, v_j) \notin E \end{cases} \quad (1)$$

Esta representación requiere un espacio en memoria de $\mathcal{O}(V^2)$, lo que la hace altamente ineficiente para **grafos dispersos** ($|E| \ll |V|^2$). Sin embargo, permite consultas de existencia de aristas en tiempo constante $\mathcal{O}(1)$.

1.3.2. Lista de Adyacencia

Asocia a cada vértice $v \in V$ una lista de sus vecinos directos. Su consumo espacial es de $\mathcal{O}(V + E)$, siendo la opción óptima para la gran mayoría de los algoritmos de exploración topológica.

1.3.3. Matrices Dispersas (Sparse Matrices)

En la computación científica de alto rendimiento, los grafos masivos se representan mediante esquemas de matrices dispersas como Compressed Sparse Row (CSR) o Compressed Sparse Column (CSC), optimizando el uso de memoria caché y permitiendo operaciones de álgebra lineal distribuida.

2. Algoritmos Clásicos de Caminos Mínimos y Exploración

La determinación de la trayectoria óptima entre nodos es el núcleo operativo del enrutamiento.

2.1. Algoritmo de Dijkstra

Diseñado por Edsger Dijkstra, es el algoritmo de referencia para encontrar los caminos mínimos desde un único nodo origen hacia todos los demás vértices en un grafo ponderado con pesos no negativos ($w(e) \geq 0$). La condición de relajación para una arista (u, v) con peso $w(u, v)$ se expresa como:

$$\text{si } d(v) > d(u) + w(u, v) \implies d(v) = d(u) + w(u, v) \quad (2)$$

Utilizando una cola de prioridad, la complejidad temporal del algoritmo se optimiza a $\mathcal{O}((V + E) \log V)$.

El algoritmo opera bajo el principio de avance codicioso (*greedy*), asegurando que una vez que se determina la distancia mínima a un nodo, esta es absoluta y no cambiará. Esto se logra en tres fases principales:

2.1.1. Inicialización

- Se asigna una distancia de 0 al nodo origen: $d(\text{origen}) = 0$.
- A todos los demás vértices del grafo se les asigna una distancia inicial de infinito: $d(v) = \infty$.
- Se introducen todos los nodos en una estructura de datos, idealmente una cola de prioridad (*Min-Heap*).

2.1.2. Extracción del Mínimo

En cada iteración, el algoritmo extrae de la cola el nodo u que tiene la distancia acumulada más pequeña. (En la primera iteración, este será siempre el origen).

2.1.3. Fase de Relajación

Para el nodo u extraído, el algoritmo revisa todos sus nodos vecinos v . Aquí es donde entra la ecuación principal:

$$d(v) > d(u) + w(u, v) \quad (3)$$

En términos prácticos, el algoritmo se pregunta: “*Si viajo hasta el nodo v pasando a través de mi nodo actual u , ¿el costo total (distancia hasta u + el peso de la arista entre u y v) es estrictamente menor que la distancia que actualmente tengo registrada para llegar a v ?*”

Si la respuesta es afirmativa, se relaja la arista: se actualiza $d(v)$ con el nuevo valor más bajo y se ajusta la posición de v en la cola de prioridad.

La complejidad $\mathcal{O}((V + E) \log V)$ se logra específicamente porque la cola de prioridad permite encontrar el siguiente nodo más cercano en tiempo $\mathcal{O}(\log V)$, en lugar de tener que escanear toda la lista de nodos $\mathcal{O}(V)$.

2.1.4. Implementación en Python (Estructura de Datos Pura)

Para entender la algoritmia real detrás de las librerías científicas, la mejor forma es implementarlo usando el módulo `heapq` nativo de Python, el cual actúa como nuestra cola de prioridad.

Vamos a modelar un problema clásico de enrutamiento físico de planta: calcular la ruta de cableado más corta desde un tablero de distribución hasta un motor específico, pasando por distintas canalizaciones.

```
1 import heapq
2
3 def dijkstra_optimizado(grafo, origen):
4     # 1. Inicializacion
5     # Asignamos infinito a todos los nodos, excepto al origen que es 0
```

```

6     distancias = {nodo: float('inf') for nodo in grafo}
7     distancias[origen] = 0
8
9     # Diccionario para reconstruir la ruta final
10    predecesores = {nodo: None for nodo in grafo}
11
12    # Cola de prioridad (Min-Heap). Almacena tuplas: (distancia_acumulada,
13    nodo)
14    cola_prioridad = [(0, origen)]
15
16    while cola_prioridad:
17        # 2. Extraccion del Minimo (O(log V))
18        distancia_actual, nodo_actual = heapq.heappop(cola_prioridad)
19
20        # Si extraemos una ruta obsoleta que ya fue mejorada, la ignoramos
21        if distancia_actual > distancias[nodo_actual]:
22            continue
23
24        # 3. Fase de Relajacion
25        for vecino, peso_arista in grafo[nodo_actual].items():
26            distancia_calculada = distancia_actual + peso_arista
27
28            # Condicion: si  $d(v) > d(u) + w(u, v) \Rightarrow d(v) = d(u) + w(u, v)$ 
29            if distancia_calculada < distancias[vecino]:
30                distancias[vecino] = distancia_calculada
31                predecesores[vecino] = nodo_actual
32
33            # Actualizamos la cola de prioridad con la nueva mejor
34            distancia
35            heapq.heappush(cola_prioridad, (distancia_calculada, vecino
36            ))
37
38    return distancias, predecesores
39
40 def reconstruir_ruta(predecesores, origen, destino):
41     ruta = []
42     nodo_actual = destino
43     while nodo_actual is not None:
44         ruta.insert(0, nodo_actual)
45         nodo_actual = predecesores[nodo_actual]
46     return ruta if ruta[0] == origen else []
47
48 # --- Ejemplo de Uso Practico ---
49 # Modelado de una red de canalizaciones (los pesos son metros de cable/
50 # tuberia)
51 red_planta = {
52     'Tablero Principal': {'Subestacion A': 12, 'Panel de Control': 18},
53     'Subestacion A': {'Tablero Principal': 12, 'Motor (460V)': 45, '
54     Generador': 20},
55     'Panel de Control': {'Tablero Principal': 18, 'Generador': 15, 'Motor

```



```

51     (460V)': 22},
52     'Generador': {'Subestacion A': 20, 'Panel de Control': 15, 'Motor (460V)': 10},
53     'Motor (460V)': {'Subestacion A': 45, 'Panel de Control': 22, 'Generador': 10}
54 }
55 nodo_inicio = 'Tablero Principal'
56 nodo_fin = 'Motor (460V)'
57
58 distancias_minimas, arbol_predecesores = dijkstra_optimizado(red_planta,
59     nodo_inicio)
60 ruta_optima = reconstruir_ruta(arbol_predecesores, nodo_inicio, nodo_fin)
61 print(f"Ruta optima trazada: {' -> '.join(ruta_optima)}")
62 print(f"Longitud total de cableado requerido: {distancias_minimas[nodo_fin]} metros")

```

Listado de código 1: Implementación del algoritmo de Dijkstra optimizado

2.1.5. Análisis del Caso Práctico

Al ejecutar este código, el algoritmo evalúa que la ruta directa desde la subestación al motor requiere demasiado material ($12 + 45 = 57$ metros), y a través del mecanismo de relajación, detecta que triangular pasando por el **Panel de Control** y el **Generador** reduce significativamente el tendido eléctrico ($18 + 15 + 10 = 43$ metros).

2.2. Algoritmo A* (A-Star)

Introducido por Hart, Nilsson y Raphael, el algoritmo A* introduce una función de evaluación $f(n) = g(n) + h(n)$ para determinar el orden de exploración, donde $h(n)$ es una heurística que estima el costo restante más bajo desde el nodo n hasta el destino final. El algoritmo A* (pronunciado “A-estrella”) representa la evolución natural de Dijkstra cuando poseemos información espacial o geométrica sobre nuestro problema.

Mientras que Dijkstra explora a ciegas en todas las direcciones como una onda de agua que se expande circularmente, A* utiliza la heurística para “estirar” esa onda y dirigirla hacia el objetivo, ignorando los caminos que se alejan de la meta.

2.2.1. Ampliación Teórica: Anatomía de la Ecuación $f(n) = g(n) + h(n)$

Para lograr esta búsqueda dirigida, A* evalúa cada nodo basándose en tres valores fundamentales:

- **$g(n)$ (Costo Real Acumulado):** Es exactamente lo mismo que calcula Dijkstra. Representa el costo (distancia, metros de cable, tiempo) que ya hemos gastado para viajar desde el origen hasta el nodo actual n .

- $h(n)$ (**Heurística - El Faro**): Es una estimación o “suposición educada” del costo que falta para llegar desde el nodo n hasta el destino final. Para que el algoritmo funcione y garantice la ruta matemáticamente óptima, esta heurística debe ser *admisible*; es decir, debe ser optimista y nunca sobrestimar el costo real.
- $f(n)$ (**Costo Total Estimado**): Es la suma de los dos anteriores y dicta la prioridad:

$$f(n) = g(n) + h(n) \quad (4)$$

La cola de prioridad de A^* siempre extraerá el nodo con el $f(n)$ más bajo.

2.2.2. Heurísticas más comunes en ingeniería

1. **Distancia Euclidiana:** Se usa cuando el movimiento es libre en cualquier ángulo (línea recta).

$$\text{Fórmula: } \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

2. **Distancia Manhattan:** Se usa cuando el movimiento está restringido a una cuadrícula ortogonal (como el tendido de bandejas portacables en los techos de una planta o el movimiento de montacargas en pasillos).

$$\text{Fórmula: } |x_2 - x_1| + |y_2 - y_1|$$

2.2.3. Implementación en Python (Ruteo Espacial en Cuadrícula)

Para ilustrar el poder de la heurística $h(n)$, implementaremos A^* en un escenario espacial.

Imagina que necesitamos trazar la ruta de una tubería de control en el plano 2D de una planta industrial. El plano es una cuadrícula donde el punto $(0, 0)$ es el tablero de control, el objetivo está en $(4, 5)$ (donde se ubica un motor de 460V), y existen áreas restringidas (muros, fosas o maquinaria pesada) que la tubería no puede atravesar.

```

1 import heapq
2
3 # 1. Definición de la función Heurística h(n)
4 # Usaremos la Distancia Manhattan ya que la tubería solo puede girar a 90
   grados
5 def heuristica_manhattan(nodo_actual, destino):
6     x1, y1 = nodo_actual
7     x2, y2 = destino
8     return abs(x1 - x2) + abs(y1 - y2)
9
10 def algoritmo_a_star(cuadrícula, inicio, fin):
11     filas = len(cuadrícula)
12     columnas = len(cuadrícula[0])
13

```

```

14 # Cola de prioridad que almacena tuplas: (f_score, g_score, (x, y))
15 cola_prioridad = [(0, 0, inicio)]
16
17 # Diccionarios para rastrear los costos g(n) y reconstruir la ruta
18 g_scores = {inicio: 0}
19 predecesores = {inicio: None}
20
21 # Movimientos ortogonales permitidos: Arriba, Abajo, Izquierda, Derecha
22 movimientos = [(-1, 0), (1, 0), (0, -1), (0, 1)]
23
24 while cola_prioridad:
25     # Extraemos el nodo con el f(n) mas bajo
26     f_actual, g_actual, nodo_actual = heapq.heappop(cola_prioridad)
27
28     # Si llegamos al motor de 460V, terminamos prematuramente (ventaja
    de A*)
29     if nodo_actual == fin:
30         break
31
32     x, y = nodo_actual
33
34     # 2. Fase de Exploracion de Vecinos
35     for dx, dy in movimientos:
36         vecino_x, vecino_y = x + dx, y + dy
37         vecino = (vecino_x, vecino_y)
38
39         # Verificamos que el vecino este dentro del plano y no sea un
    obstaculo (1)
40         if 0 <= vecino_x < filas and 0 <= vecino_y < columnas:
41             if cuadrícula[vecino_x][vecino_y] == 1:
42                 continue # Es un obstaculo fisico, lo ignoramos
43
44             # Asumimos un costo de 1 metro por cada celda avanzada
45             g_calculado = g_actual + 1
46
47             # Si encontramos un camino mas corto hacia este vecino,
    relajamos
48             if vecino not in g_scores or g_calculado < g_scores[vecino
    ]:
49                 g_scores[vecino] = g_calculado
50
51                 # f(n) = g(n) + h(n)
52                 f_calculado = g_calculado + heuristica_manhattan(vecino
    , fin)
53
54                 predecesores[vecino] = nodo_actual
55                 heapq.heappush(cola_prioridad, (f_calculado,
    g_calculado, vecino))
56
57     # 3. Reconstruccion de la ruta

```

```

58     ruta = []
59     nodo_paso = fin
60     while nodo_paso is not None:
61         ruta.append(nodo_paso)
62         nodo_paso = predecesores.get(nodo_paso)
63
64     return ruta[::-1] if ruta[-1] == inicio else []
65
66 # --- Ejemplo de Uso Practico ---
67 # Plano de la planta: 0 = Espacio libre, 1 = Obstaculo (Maquinaria/
68 # Estructuras)
69 plano_planta = [
70     [0, 0, 0, 0, 0, 0],
71     [0, 1, 1, 1, 1, 0], # Muro bloqueando el paso directo
72     [0, 0, 0, 0, 1, 0],
73     [0, 1, 1, 0, 0, 0],
74     [0, 0, 0, 0, 0, 0]
75 ]
76
77 origen = (0, 0) # Tablero
78 destino = (4, 5) # Motor 460V
79
80 ruta_optima = algoritmo_a_star(plano_planta, origen, destino)
81
82 print(f"Ruta optima encontrada (Coordenadas):")
83 for paso in ruta_optima:
84     print(f" -> {paso}")
85 print(f"\nDistancia total: {len(ruta_optima) - 1} metros (unidades)")

```

Listado de código 2: Implementación del Algoritmo A* en cuadrícula

2.2.4. La Diferencia de Rendimiento

Si ejecutáramos Dijkstra en este mismo plano bidimensional, evaluaría las celdas (0,1), luego (1,0), luego (2,0) de forma circular, perdiendo tiempo analizando zonas del plano que están en la dirección opuesta al motor.

A*, gracias al cálculo de `heuristica_manhattan(vecino, fin)`, le otorga automáticamente una prioridad muy baja en la cola a cualquier celda que suponga alejarse de la coordenada (4,5), logrando converger en la solución analizando una fracción mucho menor de los nodos totales.

2.3. Algoritmo de Bellman-Ford

A diferencia de Dijkstra, el algoritmo de Bellman-Ford puede computar caminos mínimos en grafos que contienen aristas con pesos negativos. Su funcionamiento se basa en una aproximación por programación dinámica. Es vital para **detectar ciclos de costo negativo**. Su complejidad temporal es de $\mathcal{O}(V \cdot E)$.

El algoritmo de Bellman-Ford es una herramienta matemática indispensable cuando las suposiciones optimistas de Dijkstra dejan de ser válidas. Su gran diferenciador es el manejo de pesos negativos y la detección de ciclos de costo negativo.

2.3.1. Ampliación Teórica: ¿Por qué falla Dijkstra y cómo lo soluciona Bellman-Ford?

Para entender a Bellman-Ford, primero debemos entender la debilidad de Dijkstra. Dijkstra es un algoritmo codicioso (*greedy*): asume que una vez que procesa el nodo más cercano, ya ha encontrado la ruta óptima absoluta hacia él y nunca mira atrás. Si de repente aparece una arista con un “costo negativo” (por ejemplo, un tramo que genera energía en lugar de consumirla, o un arbitraje de divisas a favor), Dijkstra no recalculará los nodos ya cerrados, entregando un resultado incorrecto.

Bellman-Ford abandona la estrategia codiciosa en favor de la programación dinámica. Funciona de la siguiente manera:

- **Relajación Global:** En lugar de buscar el nodo más cercano, Bellman-Ford toma todas las aristas del grafo y aplica la ecuación de relajación sobre cada una de ellas:

$$d(v) > d(u) + w(u, v) \quad (5)$$

- **Iteraciones Múltiples:** Aplica este proceso de relajar todas las aristas exactamente $|V| - 1$ veces (donde $|V|$ es el número total de vértices). ¿Por qué? Porque en un grafo de V nodos, el camino más largo posible sin entrar en bucles (un camino simple) tendrá como máximo $V - 1$ aristas. Al iterar $V - 1$ veces, el algoritmo garantiza que el efecto de cualquier costo negativo se propague por completo a lo largo de toda la red.
- **Detección de Ciclos (La V -ésima iteración):** Esta es su característica más potente. Una vez terminadas las $V - 1$ iteraciones, el algoritmo hace una pasada extra sobre todas las aristas. Si aún es posible seguir relajando alguna arista (es decir, encontrar una ruta aún más barata), significa que existe un ciclo de costo negativo.

Nota analítica: En un ciclo negativo, puedes girar infinitamente disminuyendo el costo total hacia $-\infty$. En redes logísticas o de potencia, esto suele indicar un error de modelado físico o una oportunidad de bucle de ganancia infinita (arbitraje).

2.3.2. Implementación en Python

Para este ejemplo, modelemos un sistema de transmisión y distribución industrial. Los nodos son subestaciones y los pesos de las aristas representan el costo energético de transmisión. Introduciremos un “peso negativo” simulando un tramo que incluye un sistema de recuperación de energía (como generación regenerativa en una cinta transportadora pesada), el cual aporta energía a la red en lugar de consumirla.

```

1 class GrafoBellmanFord:
2     def __init__(self, vertices):
3         self.V = vertices
4         self.aristas = []
5         # Diccionario para mapear nombres de nodos a indices numericos
6         self.nodos = {}
7         self.nombres = []
8
9     def agregar_nodo(self, nombre):
10        if nombre not in self.nodos:
11            self.nodos[nombre] = len(self.nombres)
12            self.nombres.append(nombre)
13
14    def agregar_arista(self, origen, destino, peso):
15        # Aseguramos que los nodos existan
16        self.agregar_nodo(origen)
17        self.agregar_nodo(destino)
18
19        u = self.nodos[origen]
20        v = self.nodos[destino]
21        self.aristas.append([u, v, peso])
22
23    def calcular_rutas(self, origen):
24        # 1. Inicializacion
25        idx_origen = self.nodos[origen]
26        distancias = {i: float("Inf") for i in range(self.V)}
27        predecesores = {i: None for i in range(self.V)}
28        distancias[idx_origen] = 0
29
30        # 2. Fase de Relajacion (Iterar V - 1 veces)
31        for _ in range(self.V - 1):
32            for u, v, w in self.aristas:
33                if distancias[u] != float("Inf") and distancias[u] + w <
34                distancias[v]:
35                    distancias[v] = distancias[u] + w
36                    predecesores[v] = u
37
38        # 3. Deteccion de Ciclos de Costo Negativo
39        # Una iteracion extra para ver si podemos seguir optimizando
40        for u, v, w in self.aristas:
41            if distancias[u] != float("Inf") and distancias[u] + w <
42            distancias[v]:
43                print(f"ALERTA: Se ha detectado un ciclo de costo negativo
44                estructural.")
45                print(f"La ruta hacia el nodo {self.nombres[v]} tiende a -
46                Infinito.")
47                return False, None, None
48
49        return True, distancias, predecesores

```

```

47     def imprimir_solucion(self, origen, distancias, predecesores):
48         print(f"--- Analisis de Rutas Optimas (Bellman-Ford) desde: {origen}
49         } ---")
49         for i in range(self.V):
50             nombre_destino = self.nombres[i]
51             costo = distancias[i]
52
53             # Reconstruir la ruta
54             ruta = []
55             nodo_actual = i
56             while nodo_actual is not None:
57                 ruta.insert(0, self.nombres[nodo_actual])
58                 nodo_actual = predecesores[nodo_actual]
59
60             print(f"Hacia {nombre_destino:15} | Costo de Energia: {costo:4}
61             | Ruta: {' -> '.join(ruta)}")
62
63 # --- Ejecucion del Modelo ---
64 # Tenemos 5 puntos de conexion en la planta
65 grafo = GrafoBellmanFord(5)
66
67 # Agregamos las conexiones (Origen, Destino, Costo/Peso)
68 # Costos positivos = caida de tension / perdida
69 grafo.agregar_arista("Generador", "Subestacion 1", 10)
70 grafo.agregar_arista("Generador", "Planta Baja", 5)
71 grafo.agregar_arista("Subestacion 1", "Planta Alta", 1)
72 grafo.agregar_arista("Planta Baja", "Subestacion 1", 3)
73 grafo.agregar_arista("Planta Baja", "Planta Alta", 9)
74 grafo.agregar_arista("Planta Baja", "Motor Retorno", 2)
75
76 # ESTA ES LA CLAVE: El Motor Retorno tiene un sistema regenerativo que
77   inyecta
78 # energia hacia la Planta Alta, modelado como un peso negativo (-6).
79 # Dijkstra fallaria estrepitosamente aqui.
80 grafo.agregar_arista("Motor Retorno", "Planta Alta", -6)
81
82 # Calculamos desde el nodo de generacion principal
83 nodo_inicio = "Generador"
84 exito, distancias_optimas, arbol_predecesores = grafo.calcular_rutas(
85     nodo_inicio)
86
87 if exito:
88     grafo.imprimir_solucion(nodo_inicio, distancias_optimas,
89                             arbol_predecesores)

```

Listado de código 3: Implementación del algoritmo de Bellman-Ford en Python

2.3.3. Análisis del Resultado

Si observas el flujo hacia la Planta Alta:

- La ruta directa más intuitiva sería **Generador** → **Subestación 1** → **Planta Alta**, con un costo de $10 + 1 = 11$.
- Sin embargo, Bellman-Ford detectará el tramo de recuperación energética. Evaluará que ir por **Generador** → **Planta Baja** → **Motor Retorno** → **Planta Alta** suma $5 + 2 - 6 = 1$.

El algoritmo identificó correctamente que derivar la energía por el circuito regenerativo es inmensamente más eficiente, algo que la heurística ciega o la codicia de otros algoritmos pasarían por alto al ver inicialmente el costo hacia “Planta Baja” y “Motor Retorno”.

2.3.4. Implementación en Python con NetworkX

Vamos a recrear una red donde una retroalimentación mal diseñada entre un panel de control y un motor crea un bucle lógico de "energía infinita" (un ciclo negativo).

```

1 import networkx as nx
2
3 # 1. Instanciamos un grafo dirigido
4 G = nx.DiGraph()
5
6 # 2. Añadimos las conexiones (Origen, Destino, Peso)
7 # Usaremos un escenario de distribución de potencia
8 G.add_weighted_edges_from([
9     ("Tablero Principal", "Subestacion A", 10),
10    ("Tablero Principal", "Panel de Control", 5),
11    ("Subestacion A", "Motor B (460V)", 1),
12    ("Panel de Control", "Subestacion A", 3),
13    ("Panel de Control", "Motor A (460V)", 2),
14
15    # Tramo regenerativo: inyecta energía a la red (peso negativo)
16    ("Motor A (460V)", "Motor B (460V)", -6),
17
18    # Falla estructural: El Motor B retroalimenta al Panel de Control,
19    # creando un ciclo cerrado con ganancia neta infinita.
20    ("Motor B (460V)", "Panel de Control", -2)
21 ])
22
23 print("--- Análisis de Red con Bellman-Ford (NetworkX) ---\n")
24
25 try:
26     # 3. Intentamos detectar si existe un ciclo negativo usando la
27     # implementación nativa
28     # NetworkX ejecutará la complejidad  $O(V * E)$  internamente
29     ciclo_negativo = nx.find_negative_cycle(G, source="Tablero Principal",
30     weight="weight")
31
32     print(" ALERTA ESTRUCTURAL: Ciclo de costo negativo detectado.")
33     print("Esto indica una anomalía topológica en el diseño de la red.")

```



```

32     print(f"Componentes atrapados en el bucle: {' -> '.join(ciclo_negativo)
33           }")
34 except nx.NetworkXError:
35     print("Red estable (sin ciclos negativos). Calculando rutas óptimas...\n")
36
37     # Si la red es estable, calculamos las rutas más cortas de forma segura
38     longitudes, rutas = nx.single_source_bellman_ford(G, source="Tablero
39     Principal", weight="weight")
40
41     destino = "Motor B (460V)"
42     print(f"Ruta óptima hacia {destino}: {' -> '.join(rutas[destino])}")
43     print(f"Costo total acumulado: {longitudes[destino]}")

```

Listado de código 4: Implementación del algoritmo de Bellman-Ford en Python

2.3.5. ¿Por qué hacerlo así? Eficiencia y Mantenibilidad:

La función `nx.find_negative_cycle()` maneja internamente las iteraciones $\mathcal{O}(V \cdot E)$ en C/Python optimizado. Solo necesitas definir el grafo. Aislamiento de la Falla: A diferencia de nuestra implementación manual anterior (que solo avisaba que existía un ciclo), `NetworkX` devuelve una lista exacta con la secuencia de nodos defectuosos (['Panel de Control', 'Motor A (460V)', 'Motor B (460V)', 'Panel de Control']). Esto es fundamental si este código fuera el backend de un panel de diagnóstico, ya que te permite señalar visualmente dónde está el error en la topología.

3. Árboles de Expansión Mínima y Flujo en Redes

3.1. Árbol de Expansión Mínima (MST)

Esquemas centrales para el diseño eficiente de infraestructuras físicas minimizando el uso de materiales y conexiones.

- **Algoritmo de Prim:** Complejidad $\mathcal{O}(E + V \log V)$. Basado en la expansión desde nodos adyacentes.
- **Algoritmo de Kruskal:** Complejidad $\mathcal{O}(E \log E)$. Enfocado en el ordenamiento de arcos de menor costo.

Los Árboles de Expansión Mínima (*Minimum Spanning Tree* o MST) representan un cambio de paradigma respecto a los algoritmos que vimos anteriormente. Mientras que Dijkstra o A* se enfocan en optimizar el viaje de un punto A a un punto B (rutas egoístas), Prim y Kruskal buscan el bien común de toda la red.

El objetivo del MST es garantizar que absolutamente todos los vértices del sistema estén interconectados, asegurando que el costo total de toda la infraestructura (la suma de todos los cables, tuberías o enlaces) sea el mínimo matemático posible, sin crear bucles o redundancias (ciclos cerrados).

3.1.1. Ampliación Teórica: Prim vs. Kruskal

Aunque ambos algoritmos llegan al mismo resultado óptimo (el mismo costo total mínimo), su filosofía de construcción es completamente opuesta, lo que los hace ideales para diferentes escenarios topológicos.

3.1.2. Algoritmo de Prim (Expansión Radial)

- **Filosofía:** “Construye desde el centro hacia afuera”.
- **Mecanismo:** Elige un nodo cualquiera como punto de partida (por ejemplo, el tablero principal). En cada iteración, mira todos los nodos que ya pertenecen a la red y busca la conexión más barata posible hacia un nodo que aún no esté conectado. El árbol crece como una mancha de aceite.
- **Caso de Uso Ideal:** Grafos densos (donde casi todos los nodos están conectados con todos), ya que su rendimiento con colas de prioridad de Fibonacci puede llegar a $\mathcal{O}(E + V \log V)$.

3.1.3. Algoritmo de Kruskal (Construcción Global)

- **Filosofía:** “Lo más barato primero, sin importar dónde esté”.
- **Mecanismo:** No crece desde un punto central. Primero, toma todas las aristas de la planta y las ordena de menor a mayor costo (por eso su complejidad dominante es $\mathcal{O}(E \log E)$ debido al algoritmo de ordenamiento). Luego, evalúa la lista de aristas ordenadas una por una: si la arista conecta dos componentes aislados, la instala; si la arista forma un circuito cerrado (ciclo) con los cables ya instalados, la descarta por redundante.
- **Caso de Uso Ideal:** Grafos dispersos y diseño de nuevas infraestructuras desde cero. Para detectar si se forma un ciclo de manera ultra rápida, utiliza una estructura de datos matemática avanzada llamada *Conjuntos Disjuntos* (*Union-Find*).

3.1.4. Implementación en Python (Algoritmo de Kruskal)

Para ilustrar este concepto aplicado a la ingeniería, supongamos que estamos estructurando la red de alimentación de potencia para un arreglo de bombas en planta. Tenemos un tablero de distribución principal y varios motores *US Motors* operando a 460V distribuidos

en la instalación. Queremos interconectarlos todos gastando la menor cantidad de metros de cable de potencia posible.

Implementaremos Kruskal, ya que nos permite ver la elegancia de la estructura UnionFind.

```
1 class UnionFind:
2     """Estructura de datos para rastrear a qué red pertenece cada nodo y
3     evitar ciclos."""
4     def __init__(self, vertices):
5         self.padre = {v: v for v in vertices}
6         self.rango = {v: 0 for v in vertices}
7
8     def encontrar(self, vertice):
9         # Compresion de ruta: conecta directamente a la raiz para busquedas
10        # mas rapidas
11        if self.padre[vertice] != vertice:
12            self.padre[vertice] = self.encontrar(self.padre[vertice])
13        return self.padre[vertice]
14
15    def unir(self, vertice1, vertice2):
16        raiz1 = self.encontrar(vertice1)
17        raiz2 = self.encontrar(vertice2)
18
19        if raiz1 != raiz2:
20            # Unimos el arbol mas pequeno debajo del mas grande (union por
21            # rango)
22            if self.rango[raiz1] > self.rango[raiz2]:
23                self.padre[raiz2] = raiz1
24            elif self.rango[raiz1] < self.rango[raiz2]:
25                self.padre[raiz1] = raiz2
26            else:
27                self.padre[raiz2] = raiz1
28                self.rango[raiz1] += 1
29            return True # Union exitosa, no formo ciclo
30        return False # Formaba un ciclo, se rechaza la conexion
31
32    def algoritmo_kruskal(nodos, aristas):
33        mst = [] # Aqui guardaremos las conexiones definitivas
34        costo_total = 0
35
36        # 1. Ordenamiento Global (O(E log E))
37        # Ordenamos todas las aristas basadas en su peso (longitud del cable)
38        aristas_ordenadas = sorted(aristas, key=lambda arista: arista[2])
39
40        # Inicializamos el sistema de conjuntos disjuntos
41        uf = UnionFind(nodos)
42
43        # 2. Seleccion Codiciosa
44        for origen, destino, peso in aristas_ordenadas:
45            # Intentamos unir los nodos. Si no forman un ciclo, la funcion
46            # retorna True.
```

```

43         if uf.unir(origen, destino):
44             mst.append((origen, destino, peso))
45             costo_total += peso
46
47             # Una red completa siempre tendra (V - 1) conexiones.
48             if len(mst) == len(nodos) - 1:
49                 break
50
51         return mst, costo_total
52
53 # --- Escenario de Aplicacion: Bateria de Bombeo ---
54 # Nodos de la red
55 equipos = [
56     "Tablero Principal",
57     "US Motors 460V - Bomba 1",
58     "US Motors 460V - Bomba 2",
59     "US Motors 460V - Bomba 3"
60 ]
61
62 % Todas las posibles canalizaciones evaluadas en los planos (Origen,
63   Destino, Metros)
64 posibles_tendidos = [
65     ("Tablero Principal", "US Motors 460V - Bomba 1", 32),
66     ("Tablero Principal", "US Motors 460V - Bomba 2", 45),
67     ("Tablero Principal", "US Motors 460V - Bomba 3", 50),
68     ("US Motors 460V - Bomba 1", "US Motors 460V - Bomba 2", 10),
69     ("US Motors 460V - Bomba 2", "US Motors 460V - Bomba 3", 15),
70     ("US Motors 460V - Bomba 1", "US Motors 460V - Bomba 3", 28)
71 ]
72
73 # Ejecutamos el algoritmo
74 conexiones_optimas, total_metros = algoritmo_kruskal(equipos,
75   posibles_tendidos)
76
77 print("--- Tendido Electrico Optimo (Arbol de Expansion Minima) ---")
78 for origen, destino, longitud in conexiones_optimas:
79     print(f"Instalar tendido: {origen} <--> {destino} ({longitud} m)")
80 print(f"\nTotal de cable de potencia requerido: {total_metros} metros")

```

Listado de código 5: Implementación de Kruskal con Union-Find para optimización de cableado

3.1.5. Análisis del Resultado Algorítmico

En este ejemplo, conectar todos los motores directamente al tablero consumiría $32 + 45 + 50 = 127$ metros de cable. El algoritmo de Kruskal ordenaría todas las opciones y seleccionaría primero el enlace corto entre la Bomba 1 y 2 (10m), luego el de la Bomba 2 y 3 (15m), y finalmente ataría este grupo completo al tablero usando el enlace de la

Bomba 1 (32 m). El resultado es una red eléctrica unificada y funcional por solo 57 metros, optimizando drásticamente la lista de materiales.

3.2. Algoritmos de Flujo Máximo

El **Teorema del Flujo Máximo - Corte Mínimo** rige la determinación de capacidades límite operativas en sistemas. Algoritmos destacados incluyen **Ford-Fulkerson** y **Edmonds-Karp** para el modelado de caudales y dimensionamiento de enlaces de red.

Los algoritmos de **Flujo Máximo** representan un cambio de paradigma total respecto a lo que veníamos analizando. Mientras que Dijkstra, A*, Prim y Kruskal se enfocan en optimizar distancias, costos o longitudes (minimización), los algoritmos de flujo se enfocan en capacidad y volumen (maximización).

En el entorno industrial, ya no nos preguntamos “¿Cuál es la ruta más corta?”, sino “¿Cuál es el volumen máximo (caudal de fluido, amperaje, datos) que puedo bombear a través de esta red sin exceder el límite físico de ningún tramo?”.

3.2.1. Ampliación Teórica: El Paradigma de la Capacidad

Para resolver este problema, la topología matemática introduce tres conceptos clave:

- **Red Capacitada:** Un grafo dirigido donde cada arista (u, v) no tiene un “costo”, sino una capacidad máxima $c(u, v)$.
- **Red Residual y Camino de Aumento:** El algoritmo de Ford-Fulkerson funciona enviando flujo desde la fuente (s) hasta el sumidero (t) a través de cualquier camino disponible. Al hacerlo, resta esa cantidad de la capacidad original, creando una “red residual”. Continúa buscando caminos en esta red residual hasta que ya no es posible empujar ni una gota más.
- **Algoritmo de Edmonds-Karp:** Es la implementación industrial estándar de Ford-Fulkerson. Su genialidad radica en que no elige los caminos al azar, sino que utiliza una Búsqueda en Anchura (BFS) para encontrar siempre el camino de aumento con la menor cantidad de saltos. Esto garantiza que el algoritmo termine en un tiempo polinomial estricto de $\mathcal{O}(V \cdot E^2)$.

3.2.2. El Teorema del Flujo Máximo - Corte Mínimo (El Cuello de Botella)

Este es uno de los teoremas más hermosos y útiles de las matemáticas discretas. Establece que **el flujo máximo que puede atravesar una red es exactamente igual a la capacidad del corte mínimo**.

Un “corte” es una línea imaginaria que divide la red en dos, separando la fuente del destino. El corte mínimo es aquel cuya suma de capacidades es la más baja de todas las divisiones posibles. En ingeniería, el corte mínimo no es más que el *cuello de botella*

crítico del sistema. Si deseas aumentar la capacidad total de tu planta, este teorema te dice exactamente qué válvulas, cables o tuberías debes reemplazar primero.

3.2.3. Implementación en Python (NetworkX)

Vamos a modelar una aplicación electromecánica e hidráulica: un arreglo de distribución de fluidos impulsado por bombas. Queremos determinar el caudal máximo en litros por segundo (L/s) que puede llegar desde un tanque principal hasta la planta de destino, dadas las restricciones físicas de las tuberías y las válvulas intermedias.

```
1 import networkx as nx
2
3 # 1. Instanciar un Grafo Dirigido
4 G = nx.DiGraph()
5
6 # 2. Definir la red con sus CAPACIDADES (no distancias)
7 # Modelamos un sistema con valvulas y equipos de bombeo. Unidades en L/s.
8 conexiones = [
9     ("Tanque Principal", "Manifold A", {"capacity": 100}),
10    ("Tanque Principal", "Manifold B", {"capacity": 50}),
11    ("Manifold A", "Bomba US Motors 1", {"capacity": 40}),
12    ("Manifold A", "Manifold B", {"capacity": 30}), # Valvula de
13    interconexion (bypass)
14    ("Manifold B", "Bomba US Motors 2", {"capacity": 60}),
15    ("Bomba US Motors 1", "Planta Destino", {"capacity": 80}),
16    ("Bomba US Motors 2", "Planta Destino", {"capacity": 70})
17 ]
18 G.add_edges_from(conexiones)
19
20 # Nodos fuente y sumidero
21 origen = "Tanque Principal"
22 destino = "Planta Destino"
23
24 # --- Calculo de Flujo Maximo (Edmonds-Karp) ---
25 # Calcula el volumen maximo que soporta el sistema
26 caudal_maximo, distribucion_flujo = nx.maximum_flow(
27     G,
28     _s=origen,
29     _t=destino,
30     flow_func=nx.algorithms.flow.edmonds_karp
31 )
32
33 print(f"--- Analisis de Capacidad del Sistema de Bombeo ---")
34 print(f"Caudal Maximo Soportado: {caudal_maximo} L/s\n")
35
36 print("Distribucion del flujo por componente (Flujo / Capacidad Limite):")
37 for u, flujos_salientes in distribucion_flujo.items():
38     for v, flujo in flujos_salientes.items():
```

```

39     capacidad_max = G[u][v]['capacity']
40     estado = "[ALERTA] SATURADO" if flujo == capacidad_max else "OK"
41     print(f"    {u:20} -> {v:20} : {flujo:3} / {capacidad_max:3} L/s [{estado}]\n")
42
43 # --- Identificacion del Cuello de Botella (Corte Minimo) ---
44 # Particiona la red para encontrar el punto de falla de capacidad
45 valor_corte, particiones = nx.minimum_cut(
46     G,
47     _s=origen,
48     _t=destino,
49     flow_func=nx.algorithms.flow.edmonds_karp
50 )
51
52 nodos_antes_del_corte = particiones[0]
53 nodos_despues_del_corte = particiones[1]
54
55 print(f"\n--- Diagnostico de Cuello de Botella (Corte Minimo) ---")
56 print(f"El limite del sistema esta dictado por los enlaces que cruzan esta division:")
57 for u in nodos_antes_del_corte:
58     for v in G.neighbors(u):
59         if v in nodos_despues_del_corte:
60             print(f"    -> Tuberia critica a redimensionar: {u} hacia {v} (
Capacidad actual: {G[u][v]['capacity']} L/s)")

```

Listado de código 6: Cálculo de Flujo Máximo y Corte Mínimo usando NetworkX

3.2.4. Análisis Estructural del Sistema

Si corres este modelo, observarás la magia de la interconexión (bypass):

- El **Manifold A** recibe 100 L/s, pero la Bomba 1 solo puede procesar 40 L/s.
- En lugar de desperdiciar esa capacidad (o causar sobrepresión), el algoritmo de Edmonds-Karp inteligentemente desvía 30 L/s a través de la válvula de interconexión hacia el **Manifold B**.
- El **Manifold B** recibe 50 L/s del Tanque + 30 L/s del bypass = 80 L/s disponibles. Sin embargo, la Bomba 2 tiene un límite de 60 L/s.
- El caudal total máximo del sistema se estabiliza matemáticamente en 100 **L/s** (40 L/s por la vía 1 + 60 L/s por la vía 2).

El análisis del **Corte Mínimo** te revelará que, a pesar de que el Tanque puede despachar 150 L/s y la Planta puede recibir 150 L/s, las tuberías de succión de las bombas (**Manifold A** → Bomba 1 y **Manifold B** → Bomba 2) suman exactamente 100 L/s de capacidad física. Ese es tu cuello de botella absoluto dictado por el teorema matemático.

4. Análisis de Centralidad y Optimización Combinatoria

4.1. Métricas de Centralidad

- **Centralidad de Intermediación (Betweenness):** Cuantifica cuántas veces un componente es un eslabón crítico para las rutas más cortas de la red.
- **PageRank:** Evalúa la dominancia estructural a partir de matrices probabilísticas de transición.

El análisis de centralidad nos obliga a cambiar la forma en la que miramos una red. Ya no estamos buscando cómo ir del punto A al punto B, sino intentando descubrir quién es el verdadero jefe operativo de la estructura.

En sistemas complejos, no todos los nodos tienen el mismo peso. Evaluar la centralidad nos permite identificar los cuellos de botella lógicos, los puntos únicos de falla (SPOF) y los componentes cuya caída aislaría por completo sectores enteros de una instalación.

4.1.1. Ampliación Teórica: Desglosando las Métricas

4.1.2. Centralidad de Intermediación (Betweenness Centrality)

Esta métrica no mide cuántas conexiones directas tiene un componente, sino cuántas veces actúa como *punto de paso*.

Matemáticamente, para un nodo v , se define como la suma de la proporción de caminos mínimos entre todos los pares de nodos s y t que pasan por v :

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}} \quad (6)$$

Donde:

- σ_{st} es el número total de caminos mínimos desde el nodo s al nodo t .
- $\sigma_{st}(v)$ es el número de esos caminos mínimos que atraviesan el nodo v .

Aplicación física: Imagina un tablero de distribución secundario. Quizás solo está conectado a un par de subestaciones y a una batería de bombas. Sin embargo, si ese tablero es la única vía para que la energía o las señales de control pasen de la nave norte a la nave sur de la planta, su *Betweenness* será altísimo. Si falla, la planta se divide en dos islas incomunicadas.

4.1.3. PageRank (Dominancia por Influencia)

Originalmente creado para clasificar páginas web, el PageRank modela un proceso estocástico (una Cadena de Markov). La premisa es: un nodo es importante si recibe conexiones de otros nodos que también son importantes.

No es un simple recuento de enlaces entrantes. Es un algoritmo iterativo que calcula la probabilidad de que un “caminante aleatorio” que navega por la red termine en un nodo específico, basándose en la matriz de transición probabilística del grafo.

Aplicación lógica: En una arquitectura de TI industrial, podrías tener docenas de módulos de software interactuando. Un servidor que aloja el ERP principal o la base de datos PostgreSQL recibirá múltiples consultas de interfaces, módulos de inventario y terminales de control. Su PageRank destacará inmediatamente su dominancia estructural sobre el resto de la red.

4.1.4. Implementación en Python (NetworkX)

Vamos a modelar una topología mixta (ciberfísica) de una planta. Analizaremos una red lógica donde los nodos representan elementos del sistema de información y control: el servidor principal del ERP, paneles de control (HMI) y los controladores de campo de varios motores de la línea de producción.

```
1 import networkx as nx
2
3 # 1. Creamos un Grafo Dirigido (el flujo de informacion y control tiene
4   direccion)
5 G = nx.DiGraph()
6
7 # 2. Definimos la topologia de la red de control industrial
8 enlaces = [
9     # El servidor central envia datos a los paneles de control de area
10     ("Servidor ERP (Odoo)", "HMI Planta Alta"),
11     ("Servidor ERP (Odoo)", "HMI Planta Baja"),
12
13     # Los paneles de control comandan a los PLC
14     ("HMI Planta Alta", "PLC Norte"),
15     ("HMI Planta Baja", "PLC Sur"),
16     ("HMI Planta Baja", "PLC Centro"),
17
18     # Los PLC controlan directamente los motores
19     ("PLC Norte", "US Motors - Eje X"),
20     ("PLC Norte", "US Motors - Eje Y"),
21     ("PLC Sur", "US Motors - Bomba Principal"),
22     ("PLC Centro", "US Motors - Compresor"),
23
24     # Un enlace critico de retroalimentacion de seguridad
25     # El compresor envia telemetria directa al HMI Planta Alta
26     ("US Motors - Compresor", "HMI Planta Alta"),
```

```

26
27     # El PLC Centro actua como rele de emergencia para el PLC Norte
28     ("PLC Centro", "PLC Norte")
29 ]
30
31 G.add_edges_from(enlaces)
32
33 # --- 3. Calculo de Metricas de Centralidad ---
34
35 # Betweenness Centrality (Quien es el puente mas critico?)
36 # endpoints=False ignora al origen y destino como puentes de si mismos
37 betweenness = nx.betweenness_centrality(G, endpoints=False)
38
39 # PageRank (Quien concentra la mayor dominancia/influencia estructural?)
40 # alpha=0.85 es el factor de amortiguacion estandar de Google
41 pagerank = nx.pagerank(G, alpha=0.85)
42
43 # --- 4. Analisis y Resultados ---
44 print("--- DIAGNOSTICO DE CENTRALIDAD ESTRUCTURAL ---\n")
45
46 print("[1] Top 3 Nodos - Centralidad de Intermediacion (Puentes Criticos):"
47       )
48 # Ordenamos de mayor a menor
49 top_betweenness = sorted(betweenness.items(), key=lambda x: x[1], reverse=
50     True)[:3]
51 for nodo, valor in top_betweenness:
52     print(f" -> {nodo:25} | Score: {valor:.4f}")
53
54 print("\n[2] Top 3 Nodos - PageRank (Dominancia de Red):")
55 top_pagerank = sorted(pagerank.items(), key=lambda x: x[1], reverse=True)
56 [:3]
57 for nodo, valor in top_pagerank:
58     print(f" -> {nodo:25} | Score: {valor:.4f}")

```

Listado de código 7: Cálculo de Betweenness Centrality y PageRank en NetworkX

4.1.5. Análisis del Comportamiento

Al ejecutar este código, verás una divergencia interesante:

- El **Servidor ERP** tendrá un alto *Betweenness* inicial porque toda la información fluye a través de él hacia las plantas, pero el **HMI Planta Alta** y el **PLC Centro** probablemente lo superen o igualen gracias a los enlaces de retroalimentación y relé que cruzan transversalmente la arquitectura, convirtiéndolos en eslabones únicos por donde la información debe transitar para completar el ciclo.
- En **PageRank**, los nodos terminales que acumulan flujo constante (como el **PLC Norte**, que recibe del HMI Planta Alta y del PLC Centro) o el mismo **HMI Planta**

Alta (que recibe del ERP y del Compresor) concentrarán una puntuación probabilística mucho más alta, indicando que son los componentes topológicamente más “pesados” o de mayor actividad recurrente en el sistema de control.

5. El Estado del Arte: Aprendizaje Profundo en Grafos (GNN)

El cruce entre IA topológica y matrices estocásticas se define mediante redes neuronales de paso de mensajes (Message Passing).

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (7)$$

Las Redes Convolucionales de Grafos (GCN) representan el estándar moderno para mantenimiento predictivo y modelado hiper-dimensional de topologías técnicas complejas. Las Redes Neuronales de Grafos (GNN) representan el punto exacto donde la topología matemática clásica se fusiona con los modelos predictivos multidimensionales.

Cuando aplicamos *Deep Learning* para el modelado de diagnósticos en infraestructuras físicas (como predecir el desgaste de rodamientos en una batería de bombas), los modelos tradicionales (como las redes neuronales densas o CNNs) fallan porque asumen que los datos son independientes o están en una cuadrícula perfecta.

Las GNNs, y específicamente las Redes Convolucionales de Grafos (GCN), entienden que si un motor operando a 460V comienza a vibrar anómalamente, esa vibración se propagará mecánicamente a través de las tuberías hacia los equipos adyacentes. La topología dicta la influencia.

5.0.1. Desglosando la Ecuación Matemática (Paso de Mensajes)

La ecuación siguiente captura elegantemente cómo la información fluye por la red en una sola capa de la red neuronal:

$$H^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)} \right) \quad (8)$$

Vamos a diseccionarla pieza por pieza para entender su lógica geométrica:

- $H^{(l)}$ (**Matriz de Características**): Es el estado actual de tus nodos. Si tienes 5 motores y estás midiendo 2 variables (temperatura y vibración), esta es una matriz de 5×2 .
- $W^{(l)}$ (**Matriz de Pesos**): Son los parámetros entrenables. Es la parte que la inteligencia artificial ajusta durante el entrenamiento para aprender qué características son realmente importantes para predecir un fallo.

- $\tilde{A}$ (**Matriz de Adyacencia con Autoenlaces**): Normalmente, la matriz de adyacencia A solo te dice quién está conectado con quién. Pero al actualizar el estado de un equipo, no solo te importa lo que le pasa a sus vecinos, ¡también te importa su propio estado actual! Por eso se le suma la matriz identidad ($\tilde{A} = A + I$).
- $\tilde{D}^{-\frac{1}{2}} \dots \tilde{D}^{-\frac{1}{2}}$ (**Normalización Simétrica**): Este es el secreto matemático para que la red no colapse. Si un colector principal está conectado a 10 bombas, al sumar la información de todas ellas, sus valores numéricos explotarían en comparación con una bomba aislada. Dividir por la raíz cuadrada de los grados ($\tilde{D}$) de ambos nodos conectados asegura que el flujo de información se normalice de manera justa, amortiguando la influencia de los nodos masivamente conectados.
- σ (**Activación**): Una función no lineal (como ReLU) que permite a la red aprender patrones complejos, no solo relaciones lineales simples.

5.0.2. Implementación en Python: Construyendo la Ecuación 7 desde Cero

Aunque librerías como *PyTorch Geometric* hacen esto con una sola línea de código en producción, la mejor forma de entender la arquitectura profunda es programar la matriz exacta usando tensores nativos de PyTorch.

Vamos a simular un paso de mensaje convolucional en una pequeña red de tres equipos interconectados.

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4
5 # --- 1. Definicion de la Topologia ---
6 # Grafo de 3 nodos (0: Tablero, 1: Motor A, 2: Motor B)
7 # Conexiones: Tablero <-> Motor A, Motor A <-> Motor B
8 A = torch.tensor([[0., 1., 0.],
9                   [1., 0., 1.],
10                  [0., 1., 0.]])
11
12 # Anadimos auto-enlaces (Matriz Identidad) para que cada nodo recuerde su
13   propio estado
14 I = torch.eye(3)
15 A_tilde = A + I
16
17 # --- 2. Normalizacion Simetrica ---
18 # Calculamos el grado de cada nodo (cuantas conexiones tiene, incluyendo el
19   autoenlace)
20 grados = A_tilde.sum(dim=1)
21
22 # Matriz diagonal D_tilde^(-1/2)
23 D_tilde_inv_sqrt = torch.diag(torch.pow(grados, -0.5))

```

```

23 # Calculamos el operador de paso de mensajes:  $D^{(-1/2)} * A_{\text{tilde}} * D^{(-1/2)}$ 
24 operador_convolutivo = torch.matmul(torch.matmul(D_tilde_inv_sqrt,
    A_tilde), D_tilde_inv_sqrt)
25
26 print("Operador Convolutivo Normalizado:")
27 print(np.round(operador_convolutivo.numpy(), 3))
28
29 # --- 3. Datos de los Nodos (Matriz H) ---
30 # Supongamos 2 características por equipo: [Temperatura, Nivel de Vibración]
31 H_1 = torch.tensor([[45.0, 0.1], # Tablero (frio, sin vibración)
32                     [80.0, 2.5], # Motor A (caliente, vibrando mucho)
33                     [50.0, 0.5]]) # Motor B (normal)
34
35 # --- 4. Parametros Entrenables (Matriz W) ---
36 # Proyectamos de 2 características a 3 características latentes (ocultas)
37 W_1 = nn.Parameter(torch.rand(2, 3))
38
39 # --- 5. APLICACION DE LA ECUACION (7) ---
40 #  $H^{(l+1)} = \sigma(D^{(-1/2)} A_{\text{tilde}} D^{(-1/2)} H^{(l)} W^{(l)})$ 
41
42 # a. Paso de mensajes (Los nodos mezclan su estado con sus vecinos)
43 mezcla_vecinal = torch.matmul(operador_convolutivo, H_1)
44
45 # b. Transformación lineal (Aprendizaje)
46 transformacion = torch.matmul(mezcla_vecinal, W_1)
47
48 # c. Activación no lineal (sigma)
49 H_1_mas_1 = torch.relu(transformacion)
50
51 print("\nMatriz de Características de Entrada  $H^{(1)}$ :")
52 print(H_1)
53 print("\nNuevo Estado Oculto de los Nodos  $H^{(l+1)}$ :")
54 print(H_1_mas_1.detach().numpy())

```

Listado de código 8: Paso de mensajes en una GCN usando PyTorch

5.0.3. ¿Qué está sucediendo aquí físicamente?

Si observas la `mezcla_vecinal` antes de la transformación lineal, verías que el Motor B (que originalmente estaba a 50°C y 0,5 de vibración) absorbió matemáticamente parte de las características del Motor A. La red neuronal acaba de modelar la transferencia térmica y armónica a través del acople estructural.

Al apilar múltiples capas (repitiendo esta ecuación), el nodo 0 recibe información del nodo 2 a través del nodo 1. Es decir, con k capas convolucionales, cada nodo adquiere “conciencia” geométrica de todos los componentes que están a k saltos de distancia.

Referencias

- [1] J.A. Bondy y U.S.R. Murty. *Graph Theory*. Graduate Texts in Mathematics, Springer, 2008.
- [2] T.H. Cormen, C.E. Leiserson, R.L. Rivest y C. Stein. *Introduction to Algorithms*. MIT Press, 4ta Edición, 2022.
- [3] T.N. Kipf y M. Welling. Semi-supervised classification with graph convolutional networks. *ICLR*, 2017.
- [4] L.R. Ford Jr. y D.R. Fulkerson. *Flows in Networks*. Princeton University Press, 2010.